Introduction to Machine Learning
Problem Solutions: K-NN, Kernel Methods, and Support Vector Machines

Prof. Sundeep Rangan

1. *K-NN in 1D.* Consider the following dataset consisting of one-dimensional input-output pairs:

i	1	2	3	4	5
x_i	0.8	2.2	3.6	4.1	5.5
y_i	2.0	2.5	3.5	5.0	4.5

   Compute the K-NN regression estimate at the query point $x = 3.7$ for:

   (a) $K = 1$

   (b) $K = 2$

   Solution:

   (a) The nearest neighbor to $x = 3.7$ is $x_3 = 3.6$ so the estimate is $\widehat{y} = y_3 = 3.5$.

   (b) For $K = 2$, the two nearest neighbors are $x_3 = 3.6$ and $x_4 = 4.1$ so the estimate is

$$\widehat{y} = \frac{1}{2}(y_3 + y_4) = \frac{1}{2}(3.5 + 5.0) = 4.25.$$

2. *Kernel smoother in 1D.* Consider the kernel

$$K(x_i, x) = \max\{0, 1 - |x - x'|\}$$

   (a) Draw the kernel $K(x_i, x)$ as a function of x for $x_i = 0$.

   (b) Using the same data as in Problem 1, compute the kernel smoothing estimate for $x = 3.7$

   Solution:

   (a) The kernel is a triangular function shown in Fig. 1.

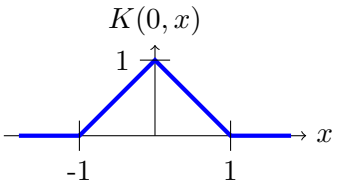

Figure 1: Problem 2. Triangular kernel

(b) Compute kernel weights:

$$K(0.8, 3.7) = \max\{0, 1 - |3.7 - 0.8|\} = 0$$
$$K(2.2, 3.7) = \max\{0, 1 - |3.7 - 2.2|\} = 0$$
$$K(3.6, 3.7) = \max\{0, 1 - |3.7 - 3.6|\} = 0.9$$
$$K(4.1, 3.7) = \max\{0, 1 - |3.7 - 4.1|\} = 0.6$$
$$K(5.5, 3.7) = \max\{0, 1 - |3.7 - 5.5|\} = 0$$

Kernel smoothing estimate:

$$\widehat{y}(3.7) = \frac{0.9 \cdot 3.5 + 0.6 \cdot 5.0}{0.9 + 0.6} = \frac{3.15 + 3.0}{1.5} = \frac{6.15}{1.5} = 4.10$$

3. *K-NN with cosine similarity.* Consider the following dataset consisting of two-dimensional input vectors and scalar output values:

i	1	2	3	4	5
x_{i1}	1	0	1	1.5	1
x_{i2}	0	1	1	1	2
y_i	2.0	2.5	3.5	5.0	4.5

Let the query point be $\boldsymbol{x} = [1.5, 1.5]$. Use cosine similarity to determine the nearest neighbors.

(a) Plot the data points $\boldsymbol{x}_i$ in 2D space. Add the query point $\boldsymbol{x}$ with a different marker.

(b) Compute the cosine similarity between the query point and each data point:

$$K(\boldsymbol{x}_i, \boldsymbol{x}) = \frac{\boldsymbol{x}_i^\intercal \boldsymbol{x}}{\|\boldsymbol{x}\| \|\boldsymbol{x}_i\|}$$

(c) Compute the K-NN regression estimate at $\boldsymbol{x}$ by averaging the y_i values of the K most similar neighbors:

- $K = 1$
- $K = 2$

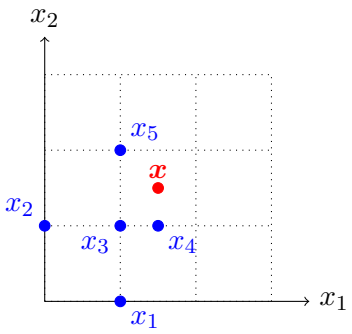

Figure 2: Problem 3. Data points (blue) and query point (red)

Solution:

(a) See Fig. 2.

(b) The norm of the query point is:

$$\|\boldsymbol{x}\| = \sqrt{1.5^2 + 1.5^2} = \sqrt{4.5} \approx 2.121$$

Now compute cosine similarities with each $\boldsymbol{x}_i$:

$$K(\boldsymbol{x}_1, \boldsymbol{x}) = \frac{1 \cdot 1.5 + 0 \cdot 1.5}{1 \cdot 2.121} = \frac{1.5}{2.121} \approx 0.707$$
$$K(\boldsymbol{x}_2, \boldsymbol{x}) = \frac{0 \cdot 1.5 + 1 \cdot 1.5}{1 \cdot 2.121} = \frac{1.5}{2.121} \approx 0.707$$
$$K(\boldsymbol{x}_3, \boldsymbol{x}) = \frac{1 \cdot 1.5 + 1 \cdot 1.5}{\sqrt{2} \cdot 2.121} = \frac{3.0}{2.121 \cdot 1.414} = 1$$
$$K(\boldsymbol{x}_4, \boldsymbol{x}) = \frac{1.5 \cdot 1.5 + 1 \cdot 1.5}{\sqrt{1.5^2 + 1^2} \cdot 2.121} = \frac{3.75}{1.803 \cdot 2.121} \approx \frac{3.75}{3.825} \approx 0.981$$
$$K(\boldsymbol{x}_5, \boldsymbol{x}) = \frac{1 \cdot 1.5 + 2 \cdot 1.5}{\sqrt{5} \cdot 2.121} = \frac{4.5}{2.121 \cdot 2.236} \approx 0.954$$

(c) For $K = 1$, the closest point is $\boldsymbol{x}_3$ so $\widehat{y} = y_3 = 3.5$. For $K = 2$, the closest points are $\boldsymbol{x}_3$ and $\boldsymbol{x}_4$ so:

$$\widehat{y} = \frac{1}{2}(y_3 + y_4) = \frac{1}{2}(3.5 + 5.0) = 4.25.$$

4. *Bias and variance.* Suppose that we have data

$$y_i = f_0(x_i) + w_i, \quad w_i \sim \mathcal{N}(0, \sigma^2),$$

where $f_0(x)$ is the unknown true function:

$$f_0(x) = x^2.$$

Suppose that at some test point $x_0 = 1.3$, we have the estimate

$$\widehat{f}(x_0) = \alpha_1 y_1 + \alpha_2 y_2,$$

with

$$x_1 = 1, \quad x_2 = 2, \quad \alpha_1 = 0.7, \quad \alpha_2 = 0.3.$$

(a) What is the bias: $\text{Bias}(x_0) = f_0(x_0) - \mathbb{E}[\widehat{f}(x_0)]$

(b) What is the variance: $\text{Var}(x_0) = \mathbb{E}[\widehat{f}(x_0) - \mathbb{E}(\widehat{f}(x_0))]^2$.

Solution:

(a) The bias is:

$$\begin{aligned}
\text{Bias}(x_0) &= f_0(x_0) - \mathbb{E}[\widehat{f}(x_0)] \\
&= f_0(x_0) - \alpha_1 \mathbb{E}(y_1) - \alpha_2 \mathbb{E}(y_1) \\
&= f_0(x_0) - \alpha_1 f_0(x_1) - \alpha_2 f_0(x_2) \\
&= x_0^2 - \alpha_1 x_1^2 - \alpha_2 x_2^2 \\
&= (1.3)^2 - (0.7)(1)^2 - (0.3)(2)^2 \approx -0.21.
\end{aligned}$$

(b) Write the estimate as

$$\widehat{f}(x_0) = \sum_i \alpha_i y_i = \sum_i \alpha_i (f_0(x_i) + w_i).$$

Since the only randomness in $\widehat{f}(x_0)$ is the due to the terms w_i, the variance is:

$$\begin{aligned}
\text{Var}(x_0) &= \mathbb{E}[\sum_i \alpha_i w_i]^2 \\
&= (\alpha_1^2 + \alpha^2)\sigma^2 = (0.3^2 + 0.7^2)\sigma^2 = 0.58\sigma^2.
\end{aligned}$$

5. *Kernel smoother implementation.* Implement the following function kernel smoother in python. Avoid using for-loops for efficient code.

```
def kernel_smoother(Xdat, ydat, X, gam=1.0):
        """
        Kernel smoother with an RBF kernel

        Parameters
        ----------
        Xdat:  (ndat,d) array of data inputs
        ydat:  (ndat,) array of data outputs
        X:  (n,d) array of query inputs

        Returns
        -------
        yhat: (n,) array of predicted outputs at the queries
        """
        ...
        return yhat
```

Solution:

One solutions is as follows:

```
def kernel_smoother(Xdat, ydat, X, gam=1.0):
        Dsq = np.sum(X[:,None,:] - Xdata[None,:,:])**2, axis=2)
        K = np.exp(-gam*Dsq/2)
        Ksum = np.sum(K,axis=1)
        yhat = K.dot(y) / Ksum
        return yhat
```

6. *Outlier detection.* Suppose one uses kernel smoothing with an RBF kernel and a query value $\boldsymbol{x}$ satisfies:

$$\sum_{i=1}^{n} K(\boldsymbol{x}, \boldsymbol{x}_i) \leq \epsilon,$$

for some small $\epsilon > 0$. This condition generally implies that the query sample $\boldsymbol{x}$ is far from all the training data samples $\boldsymbol{x}_i$. This sort of check can be used to detect *outliers* that do not follow the training data. Hence, reliable predictions cannot be made. Modify the function in Problem 5 to add an output vector `outlier` that indicates which samples are outliers. Use a default value of $\epsilon = 10^{-3}$.

Solution:

The function can be modified as:

```
def kernel_smoother(Xdat, ydat, X, gam=1.0, eps=1e-3):
        Dsq = np.sum(x[:,None,:] - xdata[None,:,:])**2, axis=2)
        K = np.exp(-gam*Dsq/2)
        Ksum = np.sum(K,axis=1)
        yhat = K.dot(y) / Ksum

        outlier = (Ksum <= epsilon).astype(int)
        return yhat, outlier
```

7. *KNN implementation.* Write a simple function in `numpy` to implement K-NN. You can use the function:

```
ind = np.argsort(A, axis=1)
```

which finds the indices for the sorting in each row. That is,

```
ind[i,k] = index of k-th smallest value in A[i,:]
```

Use the following function definition:

```
def knn(Xdat, ydat, X,k=1):
        ...
        return yhat
```

Solution:

One solution is:

```
def knn(Xdat, ydat, X,k=1):
        # Compute squared distances
        # Dsq[i,j] = distance for query i to data point j
        Dsq = np.sum((X[:,None,:] - Xdata[None,:,:])**2, axis=2)

        # Get indices of k closest neighbors
        ind = np.argsort(Dsq, axis=1)[:,:k]

        # Average over the neighbors
        n = Xdat.shape[0]
        for i in range(n):
                yhat[i] = np.mean(y[i, ind[i]])
        return yhat
```

If you wrote this, you will get full credit. But you can also implement the indexing without a loop using the following code. This code will be more efficient.

```
def knn(Xdat, ydat, X,k=1):
        # Compute squared distances
        # Dsq[i,j] = distance for query i to data point j
        Dsq = np.sum((X[:,None,:] - Xdata[None,:,:])**2, axis=2)

        # Get indices of k closest neighbors
        ind = np.argsort(Dsq, axis=1)[:,:k]

        # Get y values at the neighbors
        y_neighbors = ydat[ind]  # shape: (n_queries, k)

        # Average over neighbors
        yhat = np.mean(y_neighbors, axis=1)

        return yhat
```

8. *Margin in 1d* Consider the data set for four points with features $\mathbf{x}_i = (x_{i1}, x_{i2})$ and binary class labels $y_i = \pm 1$.

x_{i1}	0	1	1	2
x_{i2}	0	0.3	0.7	1
y_i	-1	-1	1	1

(a) Find a linear classifier that separates the two classes. Your classifier should be of the form

$$\hat{y} = \begin{cases} 1 & \text{if } b + w_1 x_1 + w_2 x_2 > 0 \\ -1 & \text{if } b + w_1 x_1 + w_2 x_2 < 0 \end{cases}$$

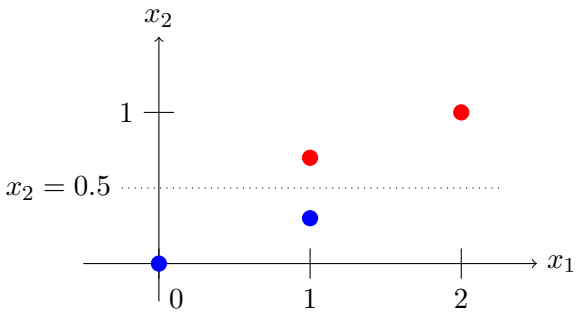

Figure 3: Problem 8: Scatter points of the data where where the red circles represent $y = 1$ and the blue circles are for the class $y = -1$

State the intercept b and weights w_1 and w_2 for your classifier. Note there is no unique answer as there are multiple linear classifiers that could separate the classes.

(b) Find the maximum γ such that

$$y_i(b + w_1 x_{i1} + w_{i2} x_{i2}) \geq \gamma, \text{ for all } i,$$

for the classifier in part (a)?

(c) Compute the margin of the classifier

$$m = \frac{\gamma}{\|\mathbf{w}\|}, \quad \|\mathbf{w}\| = \sqrt{w_1^2 + w_2^2}.$$

(d) Which samples i are on the margin for your classifier?

Solution:

(a) The best way to do this is first draw a scatter plot of the four points as shown in Fig. 3 where the red circles represent $y = 1$ and the blue circles are for the class $y = -1$.

We see that we can separate the points with the classifier,

$$\hat{y} = \begin{cases} 1 & \text{if } x_2 > 0.5 \\ -1 & \text{if } x_2 < 0.5. \end{cases}$$

We then rewrite the classifier as,

$$\hat{y} = \begin{cases} 1 & \text{if } z > 0, \\ -1 & \text{if } z < 0, \end{cases} \quad z = b + w_1 x_1 + w_2 x_2 < 0,$$

where $b = -0.5$, $w_1 = 0$ and $w_2 = 1$.

(b) Let

$$z_i = b + w_1 x_{i1} + w_2 x_{i2} = -0.5 + x_{i2}.$$

We evaluate z_i and $y_i z_i$ for each sample:

x_{i1}	0	1	1	2
x_{i2}	0	0.3	0.7	1
y_i	-1	-1	1	1
z_i	-0.5	-0.2	0.2	0.5
$y_i z_i$	0.5	0.2	0.2	0.5

Since we require $z_i y_i \geq \gamma$ for all i, the largest value of γ we can take is $\gamma = 0.2$.

(c) The margin is

$$m = \frac{\gamma}{\|\mathbf{w}\|} = \frac{0.2}{\sqrt{0+1}} = 0.2.$$

(d) The points on the margin are the ones where $z_i y_i = \gamma$. These are $(1, 0.3)$ and $(1, 0.7)$.

9. *Minimization of the hinge loss.* Consider the data set with scalar features x_i and binary class labels $y_i = \pm 1$.

x_i	0	1.3	2.1	2.8	4.2	5.7
y_i	-1	-1	-1	1	-1	1

Consider a linear classifier for this data of the form,

$$\hat{y} = \begin{cases} 1 & z > 0 \\ -1 & z < 0, \end{cases} \qquad z = x - t,$$

where t is a threshold. For each threshold t, let $J(t)$ denote the sum hinge loss,

$$J(t) = \sum_i \epsilon_i, \quad \epsilon_i = \max(0, 1 - y_i z_i).$$

(a) Write a short python program to plot $J(t)$ vs. t for 100 values of t in the interval $t \in [0, 5]$.

(b) Based on the plot, what is one value of t that minimizes $J(t)$.

(c) For the value of t in part (b), find the corresponding slack variables ϵ_i.

(d) Which samples i violate the margin ($\epsilon_i > 0$) and which samples i are misclassified ($\epsilon_i > 1$).

Solution:

(a) You can compute and plot $J(t)$ with the following code. The figure is shown in Fig. 4.

```
x = np.array([0,1.3,2.1,2.8,4.2,5.7])
y = np.array([-1,-1,-1,1,-1,1])
tvals = np.linspace(0,6,100)
J = []
for t in tvals:
    z = x-t
    Jt = np.sum(np.maximum(0,1-y*z))
    J.append(Jt)
```

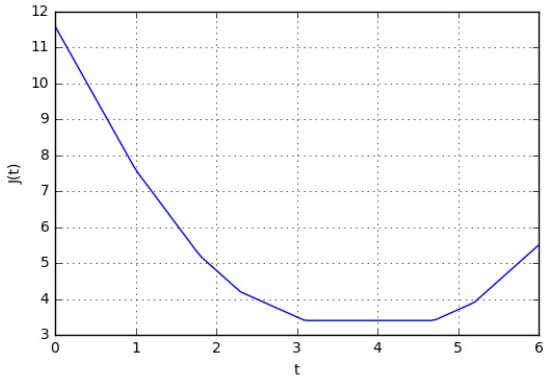

Figure 4: The objective function $J(t)$

```
J = np.array(J)

plt.plot(tvals, J)
plt.xlabel('t')
plt.ylabel('J(t)')
plt.grid()
plt.savefig('Jt.jpg')
```

(b) From Fig. 4, we see that $t = 4$ is a minimizer. In fact, $J(t)$ is flat around the minima, so other values of t close to $t = 4$ would also minimize the function.

(c) We can compute the slack variables by the python code:

```
t = 4
z = x-t
eps = np.maximum(0, 1-y*z)
eps

>> array([ 0. ,  0. ,  0. ,  2.2,  1.2,  0. ])
```

(d) We see that $\epsilon_i > 1$ for samples $i = 3, 4$ so both of these samples will be misclassified (and violate the margin).

10. *Images as matrices.* Consider an image recognition problem, where an image $\mathbf{X}$ and filter $\mathbf{W}$ are 4×4 matrices:

$$\mathbf{X} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad \mathbf{W} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

(a) Recall that in linear classification, the 4×4 image matrices $\mathbf{X}$ and $\mathbf{W}$ can be represented as 16-dimensional vectors, $\mathbf{x} = \text{vec}(\mathbf{X})$ and $\mathbf{w} = \text{vec}(\mathbf{W})$ by stacking the columns of the matrices vertically. What are $\mathbf{x}$ and $\mathbf{w}$ for the matrices above.

(b) What is the inner product $z = \mathbf{w}^T\mathbf{x}$.

(c) What is the inner product $z = \mathbf{w}^T\mathbf{x}_{\text{right}}$ where $\mathbf{x}_{\text{right}}$ is the vector corresponding to the matrix $\mathbf{X}$ right shifted by one pixel with the left column filled with zeros.

(d) What is the inner product $z = \mathbf{w}^T\mathbf{x}_{\text{left}}$ where $\mathbf{x}_{\text{left}}$ is the vector corresponding to the matrix $\mathbf{X}$ left shifted by one pixel with the right column filled with zeros.

(e) Write the python command that can covert a 4×4 image matrix, `Xmat` to the 16-dimensional vector, `x`. What is the python command to go from `x` to `Xmat`.

Solution:

(a) Going down the columns of $\mathbf{X}$ and $\mathbf{W}$, the vectors are:

$$\mathbf{x} = [0,0,0,0,\ 0,0,0,0,\ 0,1,1,1,\ 0,0,0,0]$$
$$\mathbf{w} = [0,0,0,0,\ 0,1,1,0,\ 0,1,1,0,\ 0,0,0,0]$$

(b) The inner product will be

$$z = \mathbf{w}^T\mathbf{x} = \sum_{i=1}^{16} x_i w_i.$$

Since x_i and $w_i = 0$ or 1, $x_i w_i = 1$ only on the pixels where $x_i = w_i = 1$. Hence $z = \mathbf{w}^T\mathbf{x}$ is the number of pixels where two images overlap. Thus, we have $z = 2$.

(c) If $\mathbf{X}$ is shifted to the right we have

$$\mathbf{X}_{\text{right}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

This has no overlap with $\mathbf{W}$, so $z = \mathbf{w}^T\mathbf{x}_{\text{right}} = 0$.

(d) If $\mathbf{X}$ is shifted to the left we have

$$\mathbf{X}_{\text{left}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

This overlaps with two pixels in $\mathbf{W}$. Hence, $z = \mathbf{w}^T\mathbf{x}_{\text{left}} = 2$.

(e) The python commands are `x = Xmat.ravel()` and `Xmat = x.reshape([4,4])`.

11. Consider the data set with scalar features x_i and binary class labels $y_i = \pm 1$.

x_i	0	1	2	3
y_i	1	-1	1	-1

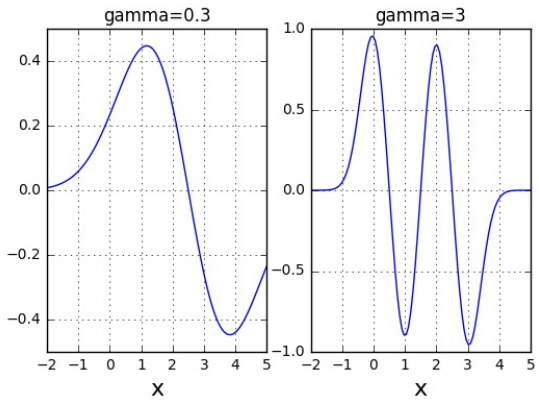

Figure 5: The function z in Problems 11(a) and (b)

A support vector classifier is of the form

$$\hat{y} = \begin{cases} 1 & z > 0 \\ -1 & z < 0, \end{cases} \quad z = \sum_i \alpha_i y_i K(x_i, x),$$

where $K(x, x')$ is the radial basis function, $K(x, x') = e^{-\gamma(x-x')^2}$, and $\gamma > 0$ and $\boldsymbol{\alpha} = [\alpha_1, \ldots, \alpha_4]$ are parameters of the classifier.

(a) Use python to plot z vs. x and $\hat{y}$ vs. x when $\gamma = 3$ and $\boldsymbol{\alpha} = [0, 0, 1, 1]$.

(b) Repeat (a) with $\gamma = 0.3$ and $\boldsymbol{\alpha} = [1, 1, 1, 1]$.

(c) Which classifier makes more errors on the training data.

Solution:

(a) You can use the python code below. Note the use of `meshgrid` command. The matrix `xpmat` has rows equal to `xp[j]` and `xmat` has columns equal to `x[i]`, Therefore, the matrix `dist` has elements `dist[i,j] = (x[i]−xp[j])**2`, which are the squared distances need to compute z.

```
x = np.array([0,1,2,3])
y = np.array([1,−1,1,−1])

def plot_zrbf(x,y,a,gam):
    xp = np.linspace(−2,5,100)
    xpmat,xmat = np.meshgrid(xp,x)
    dist = (xpmat−xmat)**2
    z = (y*a).dot(np.exp(−gam*dist))
    plt.plot(xp,z)
    plt.grid()
    plt.xlabel('x',fontsize=16)
```

```
plt.subplot(1,2,1)
a = np.array([0,0,1,1])
plot_zrbf(x,y,a,gam=0.3)
plt.title('gamma=0.3')

plt.subplot(1,2,2)
a = np.array([1,1,1,1])
plot_zrbf(x,y,a,gam=3)
plt.title('gamma=3')

plt.savefig('rbf.png')
```

The result function z is plotted in the left of Fig. 5.

(b) The function z for $\gamma = 3$ is plotted in the right of Fig. 5.

(c) The classifier takes $\hat{y}_i = \text{sign}(z_i)$. The resulting values are shown in the table below. We see that the classifier in part (b) makes no errors. Since it uses a higher γ it is able to fit the data better.

x_i	0	1	2	3
y_i	1	-1	1	-1
$\hat{y}_i = \text{sign}(z_i)$ for (a)	1	1	1	-1
$\hat{y}_i = \text{sign}(z_i)$ for (b)	1	-1	1	-1